Lecture 14: Neural Networks

A neuron you already know, stacked

Predict-first: where does logistic regression give up?

You have data in two interleaving crescents. Logistic regression (L11) is your baseline classifier.

Predict: draw the *best straight boundary* you can. What train accuracy can logistic regression reach here?

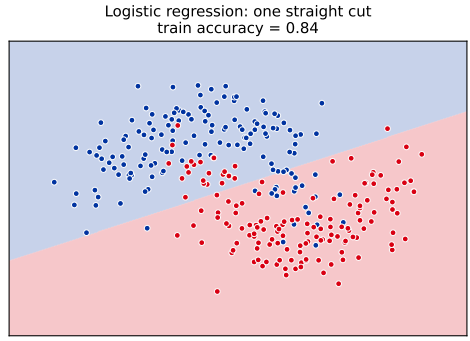
Predict-first: where does logistic regression give up?

You have data in two interleaving crescents. Logistic regression (L11) is your baseline classifier.

Predict: draw the *best straight boundary* you can. What train accuracy can logistic regression reach here?

It tops out around **0.84**. One straight cut cannot follow two curved classes – no matter how you tilt it.

We need a model whose decision boundary can **bend**. That model is a **neural network** – and its atom is a neuron you have already trained.



Outline

Why neural networks?

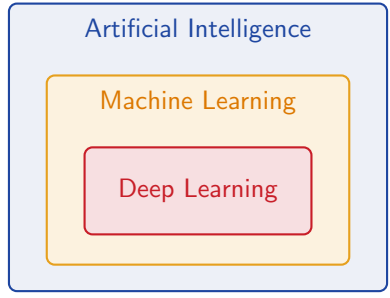
What deep learning is, when it wins, and the one idea behind it all: the model learns its own features.

What is deep learning?

Deep learning is a subfield of machine learning built on *artificial neural networks* with many layers.

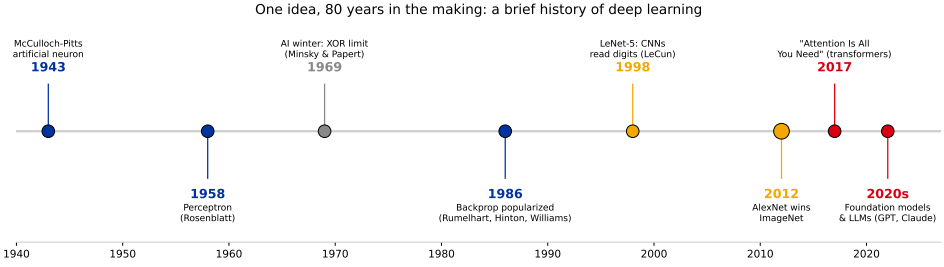
Neural nets are not new (1950s–70s). What changed:

- **Hardware** – GPUs train huge nets fast.
- **Data** – large labelled datasets exist.
- **Architectures** – specialised nets for images / text.
- **Training tricks** – better optimisation and regularisation.



One idea, 80 years in the making

Neural networks are not new - the pieces below span **eight decades** and two funding collapses (the *AI winters*). What flipped them from curiosity to workhorse was data, GPUs, and a few training tricks, not one single idea.



Keep two milestones in mind: the **perceptron** (a single neuron - next section) and **backprop** (how it learns - L15). This chapter rebuilds that arc from the neuron up.

When neural networks win – and when they do not

NNs shine when:

- the input is **high-dimensional raw signal** – pixels, audio, text;
- each single feature is weakly informative, only *combinations* matter;
- there is a **lot** of labelled data.

They usually lose when:

- the data is **tabular** (rows of curated columns);
- the dataset is small;
- you need cheap, interpretable models.

Two more NN advantages, whatever the data: *online learning* – keep updating on fresh data with a few more SGD steps, no refit from scratch (a tree must rebuild); and *transfer learning* – start from a net pretrained on a huge dataset and adapt it to your task (more on this later).

Reality check. On the tabular problems from earlier chapters, **gradient boosting still wins** most of the time without heavy tuning. Reach for a neural network when you have raw high-dimensional signals – not by default. (Why, next slide.)

Why trees still win on tabular data

NNs dominate images, audio, and text. On **tabular** data – the rows-and-columns problems from earlier chapters – gradient boosting (XGBoost, LightGBM) usually still wins. Why:

- **The columns are already the representation.** There is little raw structure left to learn – the whole strength of a NN is wasted.
- **Targets are often irregular** (jumps, thresholds) across feature space. Trees fit piecewise-constant splits naturally; NNs are biased toward *smooth* functions and fight sharp boundaries.
- **Mixed types, missing values, wild scales** – trees handle all of it natively; a NN needs encoding, imputation, and scaling first.
- **Robust to useless features** – a tree simply never splits on them; an MLP must *learn* to ignore them.
- **Less data and less tuning** to reach a strong baseline.

Rule of thumb. Raw high-dimensional signal (pixels, tokens, waveforms) $\rightarrow$ neural net. Curated table $\rightarrow$ start with gradient boosting.

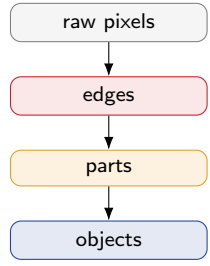
Grinsztajn, Oyallon & Varoquaux (2022), "Why do tree-based models still outperform deep learning on typical tabular data?", NeurIPS.

The one big idea: representation learning

In L01g you *hand-crafted* features: `price_per_m2`, ratios, datetime parts. A domain expert decides what matters.

A deep network does this **automatically**: early layers build simple features, later layers combine them into abstract ones. This is called **representation learning**.

Feature engineering (L01g), learned. Instead of you designing features, the network learns them from raw input – which is exactly why it needs raw signal and lots of data.



deeper layer = more abstract feature

The single neuron

One weighted sum, one nonlinearity. You have met this before – it is logistic regression wearing a diagram.

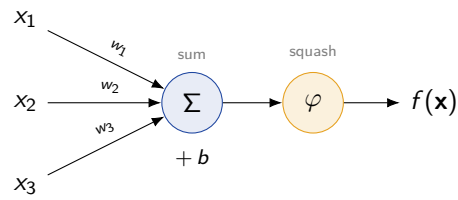
A neuron: affine transform, then activation

A single artificial neuron (a **perceptron**) does a **2-step** computation:

1. **Affine:** a weighted sum of inputs plus a bias, $z = \mathbf{w}^\top \mathbf{x} + b$.
2. **Activation:** a nonlinearity φ on that sum.

$$f(\mathbf{x}) = \varphi(w_1 x_1 + \cdots + w_p x_p + b) = \varphi(\mathbf{w}^\top \mathbf{x} + b)$$

The weights $\mathbf{w}$ and bias b are **learned**; φ is chosen.



A neuron is a regression you already know

Put the two side by side – they are *the same equation*:

Logistic regression (L11)	**A sigmoid neuron**
$\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x} + b)$	$f(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$

Only the name of the weight vector changed ($\boldsymbol{\theta} \to \mathbf{w}$). Swap the activation and you recover other models you already fit:

Activation φ	**The neuron is…**	**From**
identity ($\varphi(z) = z$)	linear regression	regression chapter
sigmoid $\sigma(z)$	**logistic regression**	L11
softmax (many outputs)	multiclass logistic regression	L11

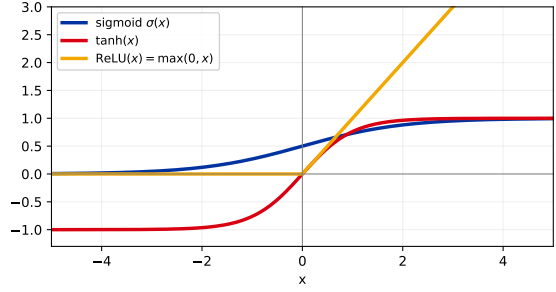
So what is a neural network? Regressions stacked on regressions – each layer's outputs become the next layer's inputs. Early layers build features; the last one is the logistic (or linear) regression you already know, run on those learned features.

Activation functions: the usual suspects

The nonlinearity is the whole point. Without it a stack of layers collapses into *one* linear model – a matrix times a matrix is still a matrix.

- **Sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$ – squashes to $(0, 1)$.
- **Tanh** $= \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$ – squashes to $(-1, 1)$, zero-centred.
- **ReLU** $\max(0, z)$ – identity for $z > 0$, flat 0 for $z < 0$.

They all add "bend". They differ in *how they pass gradients* – which decides how well a deep net actually trains. Next slide.

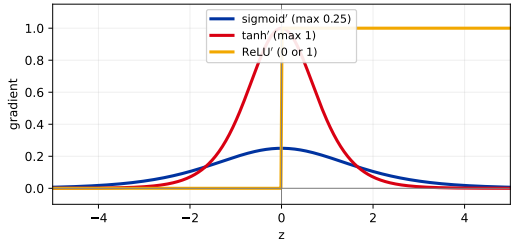


Which activation, and why

Training moves weights along *gradients* (L15). Where an activation **flattens out**, its derivative is near 0, so it passes almost no gradient back and those neurons barely learn – the **vanishing-gradient** problem.

- **Sigmoid:** derivative peaks at only 0.25 and dies at both ends; not zero-centred. Weak for hidden layers – keep it for a final probability.
- **Tanh:** zero-centred, derivative up to 1, but still saturates.
- **ReLU:** derivative is exactly 1 for $z > 0$ – gradients flow, and it is cheap. **The default for hidden layers.**

Where they go: hidden → ReLU; output → sigmoid (binary), softmax (multiclass), identity (regression).



Dead ReLU: a unit stuck at $z < 0$ has gradient 0 forever. Fixes: **Leaky ReLU** (small negative slope), **GELU** / **ELU** (smooth variants used in modern nets).

The loss is not new either

To *train* a neuron (or a whole network) we minimise empirical risk, exactly as before:

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L\Big(y^{(i)}, f(\mathbf{x}^{(i)})\Big)$$

- **Regression:** squared loss $L(y, f) = \frac{1}{2}(y - f)^2$.
- **Classification: cross-entropy** (log-loss) – $L(y, f) = -\big(y \log f + (1 - y)\log(1 - f)\big)$.

Same losses as L11. What is new is only the *model* f: instead of one linear layer, a stack of them. **How we minimise this loss – backpropagation – is L15.**

Why one neuron is not enough

A single neuron applies one nonlinearity to *one* weighted sum, so its decision boundary is a single **hyperplane** – a straight cut. That is why logistic regression stalled at 0.84 on the crescents in the opening.

Two ways out:

1. **Transform the features** by hand so the classes *become* linearly separable (e.g. Cartesian $\rightarrow$ polar). This is feature engineering.
2. **Let the model learn the transform** – put a layer of neurons *before* the output neuron. This is a neural network.

The trap. You cannot fix a curved problem by tuning one straight line harder. You need a *new feature space* – and hidden layers build one for free.

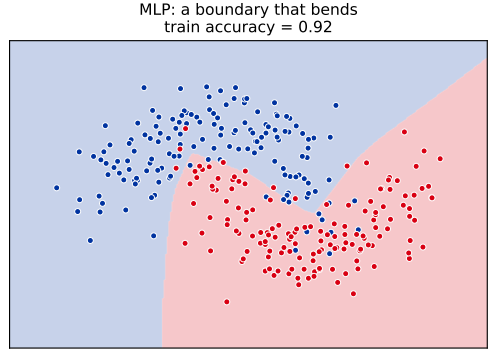
From one neuron to a network

Add a hidden layer, and the boundary starts to bend.

A hidden layer bends the boundary

Put a layer of neurons *between* input and output. Each hidden neuron learns its own weighted-sum-plus-activation – a new learned feature. The output neuron then draws a straight boundary *in that learned feature space*, which is curved back in the original space.

Same crescents, same data, **one hidden layer** of 32 ReLU neurons: **accuracy** 0.84 $\rightarrow$ 0.92, and the boundary follows the curves.



Why it works: XOR becomes separable

The smallest problem one neuron *cannot* solve. Class 1 is the diagonal $\{(0,1),(1,0)\}$, class 0 the other – no straight line splits them.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Give it **2 ReLU hidden neurons**, hidden $= \max(0, \mathbf{W}\mathbf{x} + \mathbf{b})$ with

$$\mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}.$$

Each input maps to a new point $\mathbf{z} = (z_1, z_2)$:

$\mathbf{x}$	$\mathbf{W}\mathbf{x} + \mathbf{b}$	$\mathbf{z} = \max(0, \cdot)$
$(0,0)$	$(0,-1)$	$(0,0)$
$(0,1)$	$(1,0)$	$(1,0)$
$(1,0)$	$(1,0)$	$(1,0)$
$(1,1)$	$(2,1)$	$(2,1)$

The two class-1 points now sit *on top of each other* at $(1,0)$, away from the class-0 points $(0,0)$ and $(2,1)$. A single output neuron $\operatorname{sign}(z_1 - 2z_2 - 0.5)$ splits them with one straight cut.

The hidden layer did not classify – it **re-coordinatised**. Representation learning on four points: build a space where the old straight-line model works.

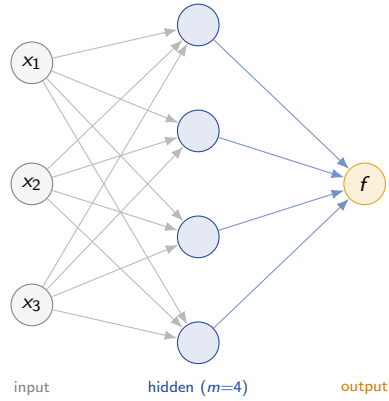
Single-hidden-layer network

A network with one hidden layer has:

- a **hidden layer** of m neurons, each doing affine + activation;
- an **output layer** of one or more neurons.

Inputs are fed in on the left; computation flows left to right. This is a **forward pass**.

Hidden neuron j: $z_j = \varphi(\mathbf{w}_j^\top \mathbf{x} + b_j)$.
Output: $f = \varphi_{\text{out}}(\mathbf{u}^\top \mathbf{z} + c)$.

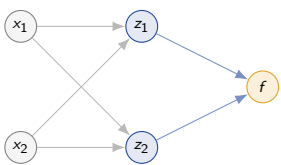


Worked forward pass, by hand

Tiny net: 2 inputs, 2 sigmoid hidden neurons, 1 sigmoid output.

Input $\mathbf{x} = (2, -1)$. Weights:

$$\mathbf{w}_1 = (0.5, -0.5),\ b_1 = 0$$
$$\mathbf{w}_2 = (-0.3,\ 0.8),\ b_2 = 0.1$$
$$\mathbf{u} = (1.0, -1.0),\ c = 0.2$$



Hidden layer (affine, then σ):

$$z_{1,\text{in}} = 0.5(2) + (-0.5)(-1) + 0 = 1.5 \ \Rightarrow\ z_1 = \sigma(1.5) = 0.82$$
$$z_{2,\text{in}} = -0.3(2) + 0.8(-1) + 0.1 = -1.3 \ \Rightarrow\ z_2 = \sigma(-1.3) = 0.21$$

Output neuron (affine, then σ):

$$f_{\text{in}} = 1.0(0.82) + (-1.0)(0.21) + 0.2 = 0.81$$
$$f_{\text{out}} = \sigma(0.81) = \mathbf{0.69}$$

The net predicts $P(y{=}1 \mid \mathbf{x}) = 0.69$. Every arrow is one multiply-add; every circle one σ. That is all a forward pass is.

The same forward pass, in matrix form

Doing each neuron separately is tidy for two of them and hopeless for a thousand. A whole layer's affine step is really **one matrix-vector product**. Stack the hidden neurons' weight vectors as the **rows** of a matrix $\mathbf{W}$:

$$\mathbf{z}_{\text{in}} = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \mathbf{W} = \begin{pmatrix} 0.5 & -0.5 \\ -0.3 & 0.8 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ 0.1 \end{pmatrix}$$

Multiply **one row at a time** – each row is exactly one neuron's weighted sum:

$$\mathbf{z}_{\text{in}} = \begin{pmatrix} 0.5 \cdot 2 + (-0.5) \cdot (-1) + 0 \\ -0.3 \cdot 2 + 0.8 \cdot (-1) + 0.1 \end{pmatrix} = \begin{pmatrix} 1.5 \\ -1.3 \end{pmatrix} \xrightarrow{\sigma \text{ (elementwise)}} \begin{pmatrix} 0.82 \\ 0.21 \end{pmatrix}$$

Same numbers as the by-hand pass – just bookkept as a matrix.

Shapes must line up. If a layer has m neurons and takes d inputs, then $\mathbf{W}$ is $m \times d$, the input is $d \times 1$, and the output is $m \times 1$. A deep net is a *chain* of these matrix multiplies – which is exactly what a GPU is built to do fast.

Why this runs on a GPU

A forward (and backward) pass is a **stack of matrix multiplies** – and a matrix multiply is thousands of independent multiply-adds that do not depend on one another.

- A **CPU** has a few fast cores – great for sequential logic, wasteful here.
- A **GPU** has thousands of small cores running the *same* operation on different data at once – exactly the shape of a matrix multiply.

Moving training from CPU to GPU cut times by more than $10\times$ and made deep nets practical – one of the "what changed" reasons from the opening.

Rule of thumb. Big dense/conv layers on lots of data $\Rightarrow$ a GPU is a 10–$100\times$ speedup. For the tiny nets in this course a laptop CPU is fine – the practical runs in minutes.

TPUs (Google) push this further: chips built only for neural-net matrix math.

Multiclass: a softmax output layer

For g classes, use g output neurons and apply **softmax** across them:

$$f_{\text{out},k} = \frac{e^{f_{\text{in},k}}}{\sum_{k'=1}^{g} e^{f_{\text{in},k'}}}$$

This returns a probability distribution over classes (sums to 1).

Same softmax as L11 multiclass logistic regression – now sitting on top of *learned* hidden features instead of raw inputs.

Mini example. Output logits (2.0, 1.0, 0.1):

$$e^{\cdot} = (7.39,\ 2.72,\ 1.11), \quad \sum = 11.22$$

$$\text{softmax} = (0.66,\ 0.24,\ 0.10)$$

Predicted class = argmax = class 1, with confidence 0.66. Softmax is a smooth, differentiable "argmax".

Deep feedforward networks

Stack more hidden layers. Why depth helps, and what it costs.

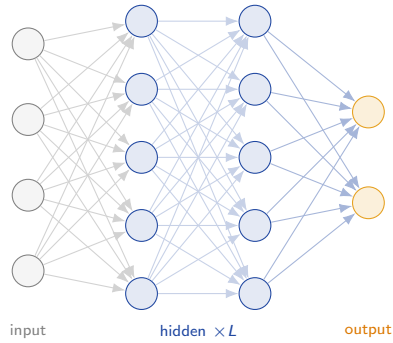
Stacking layers: the multilayer perceptron

Allow **any number L of hidden layers**. Information flows input $\rightarrow$ hidden$_1$ $\rightarrow \cdots \rightarrow$ output, with no loops: a **feedforward** network, or **MLP**.

Each layer is one affine map followed by a nonlinearity. In vector form, layer l:

$$\mathbf{a}^{(l)} = \varphi\Big(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\Big)$$

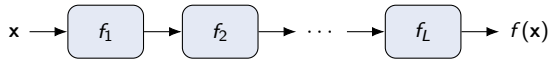
(An MLP is just a long composition of these – a fancy function. This matrix form is what we differentiate in L15.)



A network is just a composition of functions

Name each layer a function, $f_l(\mathbf{a}) = \varphi\big(\mathbf{W}^{(l)}\mathbf{a} + \mathbf{b}^{(l)}\big)$. Then the whole network is those functions **chained**:

$$f(\mathbf{x}) \;=\; \big(f_L \circ f_{L-1} \circ \cdots \circ f_1\big)(\mathbf{x}).$$



Why the nonlinearity matters. Delete every φ and the chain collapses: $\mathbf{W}^{(L)}\cdots\mathbf{W}^{(1)}\mathbf{x}$ is *one* matrix times $\mathbf{x}$ – a single linear model, however deep. The φ's are what stop the layers merging.

Why backprop is easy. The derivative of a composition is the *product* of the layers' derivatives – the **chain rule**. Read it right-to-left and that *is* backprop (L15). A 100-layer net needs no new calculus.

Why add more layers?

Each layer composes the features of the layer below into more abstract ones. Depth buys **expressivity**:

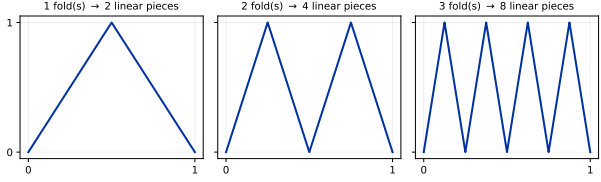
- a shallow-but-wide net and a deep net can represent the same functions in principle,
- but for many functions, a **deep** net needs exponentially *fewer* neurons than a shallow one (Montúfar et al., 2014).

Intuition: deeper layers can reuse and combine the boundaries built below, carving complex decision regions from simple pieces.

Misconception guard. The *universal approximation theorem* says one hidden layer can approximate any continuous function – but possibly with an absurd number of neurons, and it says *nothing* about whether you can **train** it or whether it will **generalise**. Depth is the practical win.

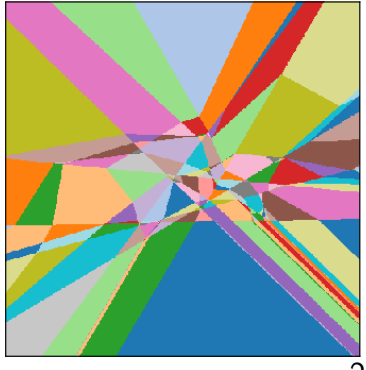
Depth folds space

A ReLU layer **folds** the input space along its kinks. Layers above then reuse the same boundary on both sides of a fold, so folds **compound** – the number of linear regions grows *exponentially* with depth.



Each fold doubles the pieces: a depth-k net expresses 2^k linear pieces from $O(k)$ neurons. A shallow net would need 2^k neurons to match.

Input plane folded into linear regions
(depth-3 ReLU net)



The catch: a lot of parameters

A modest MLP for MNIST digits ($28 \times 28 = 784$ pixels in, 10 classes out) with hidden layers of 128 and 64:

layer	weights + biases
$784 \to 128$	100,480
$128 \to 64$	8,256
$64 \to 10$	650
total	**109,386**

Training means finding good values for **all** 109,386 of them by gradient descent.

Gradient descent needs $\partial L / \partial w$ for *every* weight. Computing each one naively is hopeless.

Next (L15): backpropagation computes all 109,386 gradients in essentially one extra pass. That is what makes training networks possible.

Play with it before L15

The fastest way to build intuition for everything in this deck is to watch a net train live:

- **TensorFlow Playground** – `playground.tensorflow.org`. Add layers and neurons, switch activations, and watch the boundary bend. Try the two-spirals dataset with 1 vs 3 hidden layers, and toggle ReLU vs sigmoid.
- **3Blue1Brown**, "Neural networks" (chapters 1–3) – the best visual intuition for what the weights are doing.
- **mlu-explain.github.io/neural-networks** – a short interactive explainer.

Optional, but genuinely worth 20 minutes before the training lecture.

Recap

- A **neuron** = affine map $\mathbf{w}^\top \mathbf{x} + b$ then a nonlinearity φ. With $\varphi = \sigma$ it *is* logistic regression (L11).
- **Activations** give the model its bend: sigmoid / softmax for outputs, ReLU for hidden layers.
- One neuron draws a straight boundary; a **hidden layer** learns a new feature space so the boundary can **curve** ($0.84 \rightarrow 0.92$ on the crescents).
- A **forward pass** is just multiply-add-then-φ, layer by layer; **softmax** at the end gives multiclass probabilities (L11).
- **Depth** = learned representations, exponentially more expressive – but many parameters and no free lunch on training or tabular data.

Next (L15): how to actually learn those weights – **backpropagation** (by hand and vectorised), the PyTorch training loop, and how to keep the network from overfitting.